Lark: Language Specification

Lambda Affine Resource Kernel

Formal grammar, type rules, and operational semantics

This document is the formal specification of **Lark** (Lambda Affine Resource Kernel): a small purely functional language with Hindley-Milner type inference, affine ownership, traits, and a file-level module system.

Part I (§§ 1–6) is the *machine-checked* STLC core: types, typing rules, substitution, small-step reduction, and a type-soundness theorem, encoded and verified in an MLTT kernel with NbE. Every formula has a counterpart in `proof/lark/`.

Part II (§§ 7–13) extends to the complete language: algebraic data types, pattern matching, affine ownership, traits, modules, and the full surface grammar. These sections are paper rules derived from the implementation; they are not machine-checked.

Part III (§§ 14–16) is an *addendum*: the Phase 8 rigor extensions (repository snapshot `08/`). Parts I–II specify the language as built through Phase 7; Part III records where Phase 8 *strengthens* or *redefines* those rules — defined integer overflow, static exhaustiveness, and opt-in totality. A reader following the build chronologically should read it as a delta, exactly as the snapshots are.

Part I Machine-Checked STLC Core

---

1 Types and Contexts

Types.

$$\tau, \sigma \ ::= \ \mathsf{Int} \mid \mathsf{Float} \mid \mathsf{Bool} \mid \mathsf{Str} \mid \mathsf{Unit} \mid \tau \to \sigma$$

Contexts. A *snoc-list* of types (de Bruijn style; innermost binding first):

$$\Gamma \ ::= \ \cdot \mid \Gamma, \tau$$

Variable lookup. $\Gamma \ni \tau$ witnesses that τ is bound at some position in Γ:

$$\frac{}{\Gamma,\tau \ni \tau}\ \textsc{Here} \qquad\qquad \frac{\Gamma \ni \tau}{\Gamma,\sigma \ni \tau}\ \textsc{There}$$

2 Typing Rules (`lark-typing.lcore`)

Intrinsic encoding. A term of type $\mathsf{Expr}(\Gamma, \tau)$ *is* simultaneously an expression and a proof that it has type τ in context Γ; there is no separate typing relation. The ten constructors correspond one-to-one with the rules below.

$$\frac{}{\Gamma \vdash n : \mathsf{Int}}\ \textsc{LitInt} \qquad \frac{}{\Gamma \vdash f : \mathsf{Float}}\ \textsc{LitFlt} \qquad \frac{}{\Gamma \vdash b : \mathsf{Bool}}\ \textsc{LitBool} \qquad \frac{}{\Gamma \vdash s : \mathsf{Str}}\ \textsc{LitStr}$$

$$\frac{}{\Gamma \vdash () : \mathsf{Unit}}\ \textsc{Unit}$$

$$\frac{\Gamma \ni \tau}{\Gamma \vdash x : \tau}\ \textsc{Var} \qquad \frac{\Gamma, \tau_1 \vdash e : \tau_2}{\Gamma \vdash \lambda x{:}\tau_1.\, e : \tau_1 \to \tau_2}\ \textsc{Lam}$$

$$\frac{\Gamma \vdash f : \tau_1 \to \tau_2 \qquad \Gamma \vdash a : \tau_1}{\Gamma \vdash f\ a : \tau_2}\ \textsc{App} \qquad \frac{\Gamma \vdash v : \tau_1 \qquad \Gamma, \tau_1 \vdash b : \tau_2}{\Gamma \vdash \mathbf{let}\ x{=}v\ \mathbf{in}\ b : \tau_2}\ \textsc{Let}$$

$$\frac{\Gamma \vdash c : \mathsf{Bool} \qquad \Gamma \vdash t : \tau \qquad \Gamma \vdash e : \tau}{\Gamma \vdash \mathbf{if}\ c\ \mathbf{then}\ t\ \mathbf{else}\ e : \tau}\ \textsc{If}$$

3 Semantic Environments and Substitution (`lark-subst.lcore`)

Semantic environments. $\mathsf{Env}(\Gamma)$ maps each position in Γ to a closed term of the corresponding type:

$$\begin{aligned} \mathsf{Env}(\cdot) &= \mathbf{1} \\ \mathsf{Env}(\Gamma, \tau) &= \mathsf{Expr}(\cdot, \tau) \times \mathsf{Env}(\Gamma) \end{aligned}$$

The definition is by recursion on Γ; the product unfolds *definitionally*, which is the key to avoiding index-refinement in the kernel.

Variable lookup. $\mathsf{lookup} : \Gamma \ni \tau \to \mathsf{Env}(\Gamma) \to \mathsf{Expr}(\cdot, \tau)$ is defined by recursion on the derivation:

$$\mathsf{lookup}(\textsc{Here},\ (v; \eta)) = v \qquad \mathsf{lookup}(\textsc{There}(x),\ (v; \eta)) = \mathsf{lookup}(x,\ \eta)$$

Open semantic environments. The closed-environment $\mathsf{Env}(\Gamma)$ suffices for computing ground values but not for substituting *under a* λ. We therefore generalise to an *open* environment

$$\mathsf{Env}_\Delta(\Gamma)$$

which maps each position in Γ to an *open* term in context Δ:

$$\begin{aligned} \mathsf{Env}_\Delta(\cdot) &= \mathbf{1} \\ \mathsf{Env}_\Delta(\Gamma, \tau) &= \mathsf{Expr}(\Delta, \tau) \times \mathsf{Env}_\Delta(\Gamma) \end{aligned}$$

$\mathsf{Env}_{\cdot}(\Gamma) = \mathsf{Env}(\Gamma)$ definitionally.

Variable lookup in an open environment is identical in structure to the closed case:

$$\mathsf{lookup}_\Delta(\textsc{Here},\ (v; \eta)) = v \qquad \mathsf{lookup}_\Delta(\textsc{There}(x),\ (v; \eta)) = \mathsf{lookup}_\Delta(x,\ \eta)$$

Weakening an open environment. Given $\eta : \mathsf{Env}_\Delta(\Gamma)$, inserting a new innermost binding σ into Δ gives $\overline{\eta}_\sigma : \mathsf{Env}_{\Delta,\sigma}(\Gamma)$ by applying the kernel primitive `weaken` to each entry:

$$\overline{\eta}_\sigma \;=\; \mathsf{wk}_\sigma(\eta) \qquad \text{(see §6)}$$

Semantic substitution. $e[\eta]$ simultaneously replaces all free variables of e with the corresponding open terms in $\eta : \mathsf{Env}_\Delta(\Gamma)$, producing an open term in context Δ (`sub_open`). The ground case $\Delta = \cdot$ recovers closed substitution (`sub_ground`).

e	$e[\eta]$	Note
n, f, b, s, $()$	n, f, b, s, $()$	literals are closed
x_{HERE}	$\mathsf{fst}\,\eta$	innermost variable
x_{THERE}	$x[\mathsf{snd}\,\eta]$	skip one binder
$f\,a$	$f[\eta]\ a[\eta]$	
$\mathbf{let}\ x{=}v\ \mathbf{in}\ b$	$b[(v[\eta];\,\eta)]$	let-unrolling
$\mathbf{if}\ c\ \mathbf{then}\ t\ \mathbf{else}\ e$	$\mathbf{if}\ c[\eta]\ \mathbf{then}\ t[\eta]\ \mathbf{else}\ e[\eta]$	
$\lambda x{:}\tau_1.\,b$	$\lambda x{:}\tau_1.\,b[(x_{\mathsf{here}};\,\overline{\eta}_{\tau_1})]$	extend & weaken (see §6)

Value predicate. $\mathsf{IsVal}(\tau,\,e)$ holds when the *closed* expression $e : \mathsf{Expr}(\cdot, \tau)$ is already a value:

$$\frac{}{\mathsf{IsVal}(\mathsf{Int},\,n)}\ \textsc{ValInt} \qquad \frac{}{\mathsf{IsVal}(\mathsf{Float},\,f)}\ \textsc{ValFlt} \qquad \frac{}{\mathsf{IsVal}(\mathsf{Bool},\,b)}\ \textsc{ValBool} \qquad \frac{}{\mathsf{IsVal}(\mathsf{Str},\,s)}\ \textsc{ValStr}$$

$$\frac{}{\mathsf{IsVal}(\mathsf{Unit},\,())}\ \textsc{ValUnit} \qquad \frac{}{\mathsf{IsVal}(\tau_1 \to \tau_2,\,\lambda x.\,e)}\ \textsc{ValLam}$$

4 Small-Step Reduction (`lark-step.lcore`)

$e \to e'$ relates *closed* terms of the same type τ (in the kernel: $\mathsf{Step}(\tau, e, e')$).

Base rules:

$$\frac{}{(\lambda x.\,b)\ v\ \to\ b[(v;\,\langle\rangle)]}\ \beta \qquad \frac{}{\mathbf{if}\ \mathsf{true}\ \mathbf{then}\ t\ \mathbf{else}\ e\ \to\ t}\ \textsc{IfT} \qquad \frac{}{\mathbf{if}\ \mathsf{false}\ \mathbf{then}\ t\ \mathbf{else}\ e\ \to\ e}\ \textsc{IfF}$$

$$\frac{\mathsf{IsVal}(\tau_1,\,v)}{\mathbf{let}\ x{=}v\ \mathbf{in}\ b\ \to\ b[(v;\,\langle\rangle)]}\ \textsc{LetBeta}$$

Congruence rules (call-by-value order):

$$\frac{f \to f'}{f\,a\ \to\ f'\,a}\ \textsc{App1} \qquad \frac{\mathsf{IsVal}(\tau_1 \to \tau_2,\,f) \qquad a \to a'}{f\,a\ \to\ f\,a'}\ \textsc{App2} \qquad \frac{c \to c'}{\mathbf{if}\ c\ t\ e\ \to\ \mathbf{if}\ c'\ t\ e}\ \textsc{IfCong}$$

$$\frac{v \to v'}{\mathbf{let}\ x{=}v\ \mathbf{in}\ b\ \to\ \mathbf{let}\ x{=}v'\ \mathbf{in}\ b}\ \textsc{LetCong}$$

Remark 1 (Type preservation). $\to$ is typed: $\mathsf{Step}(\tau, e, e')$ carries τ as an explicit index, so both endpoints have the same type by construction. Type preservation requires no separate proof.

5 Multi-Step Reduction and Evaluation (`lark-preservation`)

Multi-step reduction. $e \to^* e'$ is the reflexive-transitive closure of $\to$:

$$\frac{}{e \to^* e}\ \textsc{Refl} \qquad \frac{e \to e' \qquad e' \to^* e''}{e \to^* e''}\ \textsc{Cons}$$

Big-step evaluation. $e \Downarrow v$ derives from $\to$ by requiring the endpoint to be a value:

$$\frac{\mathsf{IsVal}(\tau,\,v)}{v \Downarrow v}\ \textsc{Done} \qquad \frac{e \to e' \qquad e' \Downarrow v}{e \Downarrow v}\ \textsc{Step}$$

Main theorem.

Theorem 1 (Evaluation soundness)**.** *If* $e \Downarrow v$ *then* $\mathsf{IsVal}(\tau,\,v)$.

Proof. By induction on the derivation of $e \Downarrow v$.

- Done: $e = v$ and $\mathsf{IsVal}(\tau,\,v)$ is a direct hypothesis.

- STEP: $e \to e'$ and $e' \Downarrow v$. By the induction hypothesis on $e' \Downarrow v$, we obtain $\mathsf{IsVal}(\tau,\, v)$. □

In the kernel, this is `eval_produces_val`, proved by `indrec Eval` with motive $M(t, e, v, -) = \mathsf{IsVal}(t, v)$. The DONE case returns the `IsVal` witness directly; the STEP case passes through the induction hypothesis. The normal form of the proof is:

$$\texttt{indrec Eval}$$
$$(\lambda t\, e\, v\, \_.\ \mathsf{IsVal}(t, v))\ (\lambda t\, v\ iv.\ iv)\ (\lambda t\, e\, e'\, v\, s\, ev'\ ih.\ ih)$$

6 The `weaken` Primitive

The ELam case of `sub_open` requires lifting each entry $e_i : \mathsf{Expr}(\Delta, \tau_i)$ of $\eta : \mathsf{Env}_\Delta(\Gamma)$ to $\mathsf{Expr}(\Delta, \sigma, \tau_i)$, so that the extended environment has type $\mathsf{Env}_{\Delta,\sigma}(\Gamma)$. This operation is *weakening*:

$$\mathsf{weaken} : \forall\, \sigma\, \Delta\, \tau.\ \mathsf{Expr}(\Delta, \tau)\ \longrightarrow\ \mathsf{Expr}(\Delta, \sigma, \tau).$$

Definitional obstacle. A definition of `weaken` by `indrec Expr` within the kernel meets a definitional obstacle in the ELam case. Given $\mathsf{ELam}(\Delta, \tau_\lambda, \tau_2, \mathit{body})$ with $\mathit{body} : \mathsf{Expr}(\mathsf{ext}(\tau_\lambda, \Delta), \tau_2)$, the weakened lambda requires $\mathit{body}' : \mathsf{Expr}(\mathsf{ext}(\tau_\lambda, \mathsf{ext}(\sigma, \Delta)), \tau_2)$, whereas a recursive call $\mathsf{weaken}(\sigma, \mathsf{ext}(\tau_\lambda, \Delta), \tau_2, \mathit{body})$ produces type $\mathsf{Expr}(\mathsf{ext}(\sigma, \mathsf{ext}(\tau_\lambda, \Delta)), \tau_2)$. Unifying these requires

$$\mathsf{ext}(\sigma,\, \mathsf{ext}(\tau_\lambda,\, \Delta))\ =\ \mathsf{ext}(\tau_\lambda,\, \mathsf{ext}(\sigma,\, \Delta)),$$

which holds propositionally but not *definitionally* in the kernel.

Kernel primitive. `weaken` is therefore a kernel primitive (`TM_WEAKEN` in `eval.c`), operating directly on the `VL_INDCON` tree representing an Expr value, via a de Bruijn *cutoff* d:

- **`weaken_ctx`**(a, Γ, d) inserts type a at depth d in Γ; $d = 0$ prepends, $d > 0$ recurses under existing binders.
- **`shift_var_val`**(v, a, d) shifts a Var node; at $d = 0$ a HERE becomes THERE, at $d > 0$ only the tail context is updated.
- **`weaken_expr_val`**(e, a, d) recurses on e, updating every context-index argument and every variable reference; the ELam case increments d to protect the lambda's own parameter.

All context-index arguments are updated consistently throughout the constructed `VL_INDCON` tree, so the result has the correct type without appealing to any commutativity equation.

Syntax. In `.lcore` files:

$$\texttt{weaken}\ \sigma\ \Delta\ \tau\ e\ \ \texttt{: Expr (ext}\ \sigma\ \Delta\texttt{)}\ \tau$$

The ELam case of substitution. With `weaken` in place, the ELam row of the substitution table (§3) is:

$$\lambda x{:}\tau_1.\, b[\eta]\ =\ \lambda x{:}\tau_1.\, b[\big(x_{\mathsf{here}};\ \mathsf{wk}_{\tau_1}(\eta)\big)]$$

where $\mathsf{wk}_{\tau_1}(\eta)$ lifts each $e_i : \mathsf{Expr}(\Delta, \tau_i)$ in η to $\mathsf{Expr}(\Delta, \tau_1, \tau_i)$ via `weaken` (`weaken_semctx_d` in the source), and $x_{\mathsf{here}} = \mathsf{EVar}(\Delta, \tau_1, \text{HERE})$. No commutativity equation is required.

As a concrete instance:

$$\texttt{sub\_ground}\ (\Gamma{=}\mathsf{Int})\ (\tau{=}\mathsf{Bool}{\to}\mathsf{Bool})\ (\lambda x{:}\mathsf{Bool}.\, x)\ [42]\ =\ \lambda x{:}\mathsf{Bool}.\, x$$

(The lambda parameter remains at index 0 in the empty output context.)

Remark 2 (Definitional equality of stuck `weaken` terms). When the argument of `weaken` is neutral, the evaluator suspends and records a `SP_WEAKEN` spine entry. Two terms $\mathsf{weaken}(\sigma_1, \Delta, \tau, x)$ and $\mathsf{weaken}(\sigma_2, \Delta, \tau, x)$ at the same neutral x are definitionally equal if and only if σ_1, Δ, and τ each agree definitionally. The conversion checker (`conv_spine` in `check.c`) compares all three parameters.

Part II Full Language Specification

---

7 Extended Types

Types. The STLC core (§1) uses six base types and function types. The full language adds tuples, algebraic data types, type variables, and the affine resource type IO:

$$\tau, \sigma \ ::= \ \mathsf{Int} \mid \mathsf{Float} \mid \mathsf{Bool} \mid \mathsf{Str} \mid \mathsf{Unit} \mid \mathsf{IO} \ \mid \ \tau \to \sigma \ \mid \ (\tau_1, \ldots, \tau_n) \ \ (n \geq 2) \ \mid \ \alpha \ \mid \ T \ \mid \ T(\tau_1, \ldots, \tau_n)$$

T ranges over declared type constructors; α over type variables (lowercase names in source).

Type schemes. Let-generalisation (HM Algorithm W) produces type schemes $\forall \bar{\alpha}.\, \tau$ at top-level `fn` and `let` boundaries. At each use site the bound variables are replaced by fresh monotypes (instantiation).

Built-in types.

Type	Arity	Copy
Int, Float, Bool, Str, Unit	0	yes
IO	0	no (affine)
List(α)	1	iff Copy(α)
Result(α, β)	2	iff Copy(α) $\wedge$ Copy(β)
$(\tau_1, \ldots, \tau_n)$	$n \geq 2$	iff $\forall i.$ Copy(τ_i)

8 Affine Discipline

Lark's ownership model is *affine*: a value of a non-`Copy` type may be used at most once. Types opt into free duplication by satisfying the Copy predicate.

The `Copy` predicate. Copy(τ) is defined by structural recursion on τ:

τ	Copy(τ)	Note
Int, Float, Bool, Str, Unit	yes	built-in
$\tau \to \sigma$	yes	functions are code, not resources
$(\tau_1, \ldots, \tau_n)$	Copy(τ_1) $\wedge \cdots \wedge$ Copy(τ_n)	
α (type variable)	yes	conservative; prevents false positives
T (nullary)	yes iff `impl Copy for T` declared	
$T(\tau_1, \ldots, \tau_n)$	yes iff $T \in$ CopyTypes $\wedge\ \forall i.$ Copy(τ_i)	
IO	no	no `Copy` impl

CopyTypes is initialised to the built-in Copy types and extended by each `impl Copy for T` declaration in the program.

Affine tracking. The type checker maintains a partial map $\Omega : \mathsf{Var} \rightharpoonup \mathbb{N}$ (the *affine environment*) from variable names to use counts.

- **Bind.** A parameter $x : \tau$ or let-binding $x = e : \tau$ with $\neg$Copy(τ) is added to Ω with $\Omega(x) = 0$. Parameters with Copy(τ) are not tracked.
- **Pattern binders.** Variables introduced by match arms are not tracked; their types may be unresolved at check time and receive conservative Copy treatment.
- **Globals.** Top-level function names and built-in names are never tracked.
- **Use.** Each occurrence of tracked x increments $\Omega(x)$. $\Omega(x) > 1$ raises AffineError. $\Omega(x) = 0$ is permitted (affine values may be discarded).
- **Shadow.** A `let` rebinding a tracked name x removes the old entry and creates a new one iff the new binding type is non-Copy. This allows the IO-threading idiom **let** io = print(io, s) **in** ... without error.

9 Extended Typing Rules

The judgment $\Gamma \vdash e : \tau$ is the same as in §2. The affine constraint from §8 applies as a side condition on all rules: every tracked variable used in e must have use count ≤ 1 after checking.

Tuple introduction.

$$\frac{\Gamma \vdash e_1 : \tau_1 \qquad \cdots \qquad \Gamma \vdash e_n : \tau_n}{\Gamma \vdash (e_1, \ldots, e_n) : (\tau_1, \ldots, \tau_n)}\ \textsc{Tuple} \quad (n \geq 2)$$

Constructor introduction. Each ADT constructor C is registered in the global constructor environment Σ with a type scheme. Nullary constructors have scheme $\forall \bar{\alpha}.\ T(\bar{\alpha})$; constructors with payload have scheme $\forall \bar{\alpha}.\ \tau_1 \to \cdots \to \tau_k \to T(\bar{\alpha})$:

$$\frac{C : \forall \bar{\alpha}.\ \tau_1 \to \cdots \to \tau_k \to T(\bar{\alpha}) \ \in\ \Sigma}{\Gamma \vdash C : \tau_1[\bar{\sigma}/\bar{\alpha}] \to \cdots \to \tau_k[\bar{\sigma}/\bar{\alpha}] \to T(\bar{\sigma})}\ \textsc{Con}$$

Constructor application is curried; App from §2 handles each argument.

Pattern matching. $\mathsf{binds}(p, \tau)$ is the environment of variable bindings introduced by pattern p at type τ:

$$\begin{aligned}
\mathsf{binds}(_,\ \tau) &= \emptyset \\
\mathsf{binds}(x,\ \tau) &= \{x : \tau\} \\
\mathsf{binds}(n,\ \tau) &= \emptyset \quad \text{(literal)} \\
\mathsf{binds}(C(p_1, \ldots, p_k),\ T(\bar{\sigma})) &= \biguplus_i \mathsf{binds}(p_i,\ \tau_i[\bar{\sigma}/\bar{\alpha}]) \\
\mathsf{binds}((p_1, \ldots, p_n),\ (\sigma_1, \ldots, \sigma_n)) &= \biguplus_i \mathsf{binds}(p_i,\ \sigma_i)
\end{aligned}$$

where $\biguplus$ requires disjoint domains.

$$\frac{\begin{array}{c}\Gamma \vdash e : \tau \\ \forall i :\ \Gamma,\ \mathsf{binds}(p_i, \tau) \vdash e_i : \sigma\end{array}}{\Gamma \vdash \texttt{match}\ e\ \texttt{with}\ \overline{\mid\ p_i \Rightarrow e_i}\ \texttt{end} : \sigma}\ \textsc{Match}$$

All arms share the same result type σ (unified by HM). Arms are tested in source order; a non-exhaustive match is a runtime error.

Function declarations and let-recursion. A `fn` declaration adds its own name to scope before checking the body, enabling direct recursion:

$$\frac{\Gamma,\ f : \tau_1 \to \cdots \to \tau_k \to \rho,\ x_1 : \tau_1,\ \ldots,\ x_k : \tau_k\ \vdash\ e : \rho}{\Gamma\ \vdash\ \texttt{fn}\ f(x_1{:}\tau_1, \ldots, x_k{:}\tau_k){:}\rho = e\ \dashv\ f : \forall \bar{\alpha}.\ \tau_1 \to \cdots \to \tau_k \to \rho}\ \textsc{FnDecl}$$

Mutual recursion: all top-level `fn` declarations carrying complete type annotations are added to Γ simultaneously before any body is checked (pre-registration pass). Functions without full annotations use the single-function let-rec path above.

Operators.

$$\frac{\Gamma \vdash e_1 : \alpha \qquad \Gamma \vdash e_2 : \alpha \qquad \alpha \in \{\mathsf{Int}, \mathsf{Float}\}}{\Gamma \vdash e_1\ op\ e_2 : \alpha}\ \textsc{Arith} \quad (op \in \{+, -, *, /\})$$

$$\frac{\Gamma \vdash e_1 : \alpha \qquad \Gamma \vdash e_2 : \alpha}{\Gamma \vdash e_1\ cmp\ e_2 : \mathsf{Bool}}\ \textsc{Cmp} \quad (cmp \in \{==, \neq, <, \leq, >, \geq\})$$

$$\frac{\Gamma \vdash e_1 : \mathsf{Bool} \qquad \Gamma \vdash e_2 : \mathsf{Bool}}{\Gamma \vdash e_1\ bop\ e_2 : \mathsf{Bool}}\ \textsc{Bool} \quad (bop \in \{\texttt{and}, \texttt{or}\})$$

IO operations. IO is a pre-declared affine type with two built-in operations in Γ:

$$\mathsf{print} : \mathsf{IO} \to \mathsf{Str} \to \mathsf{IO} \qquad \mathsf{read} : \mathsf{IO} \to (\mathsf{IO}, \mathsf{Str})$$

Since $\neg\mathsf{Copy}(\mathsf{IO})$, the affine discipline ensures each IO token threads linearly through the call chain.

10 Extended Operational Semantics

The small-step relation $e \to e'$ and value predicate $\mathsf{IsVal}(\tau,\, v)$ from §4 are extended with tuples, constructor values, and pattern-match reduction.

Extended value forms.

$$\frac{\mathsf{IsVal}(\tau_i,\, v_i) \text{ for each } i}{\mathsf{IsVal}((\tau_1, \ldots, \tau_n),\, (v_1, \ldots, v_n))}\ \textsc{ValTup}$$

$$\frac{C : \forall \bar{\alpha}.\ \tau_1 \to \cdots \to \tau_k \to T(\bar{\alpha}) \ \in\ \Sigma \qquad \mathsf{IsVal}(\tau_i[\bar{\sigma}/\bar{\alpha}],\, v_i) \text{ for each } i}{\mathsf{IsVal}(T(\bar{\sigma}),\, C\, v_1 \cdots v_k)}\ \textsc{ValCon}$$

Pattern-match function. $\mathsf{match}(v, p)$ returns a substitution θ or fail:

$$\begin{aligned}
\mathsf{match}(v,\, _) &= \{\} \\
\mathsf{match}(v,\, x) &= \{x \mapsto v\} \\
\mathsf{match}(n,\, n) &= \{\} \quad \text{(same literal)} \\
\mathsf{match}(C\, v_1 \cdots v_k,\ C(p_1, \ldots, p_k)) &= \biguplus_i \mathsf{match}(v_i, p_i) \\
\mathsf{match}(v,\ C'(\ldots)) &= \mathsf{fail} \quad (C' \neq C, \text{ or arity mismatch}) \\
\mathsf{match}((v_1, \ldots, v_n),\ (p_1, \ldots, p_n)) &= \biguplus_i \mathsf{match}(v_i, p_i)
\end{aligned}$$

Match reduction.

$$\frac{\mathsf{match}(v,\, p_j) = \theta \qquad \forall i < j :\ \mathsf{match}(v,\, p_i) = \mathsf{fail}}{\texttt{match}\ v\ \texttt{with}\ \overline{\mid p_i \Rightarrow e_i}\ \texttt{end}\ \to\ e_j[\theta]}\ \textsc{MatchArm}$$

$$\frac{e \to e'}{\texttt{match}\ e\ \texttt{with}\ \ldots\ \texttt{end}\ \to\ \texttt{match}\ e'\ \texttt{with}\ \ldots\ \texttt{end}}\ \textsc{MatchCong}$$

Tuple congruence.

$$\frac{e_i \to e_i' \qquad \mathsf{IsVal}(\tau_j,\, v_j) \text{ for } j < i}{(v_1, \ldots, v_{i-1}, e_i, e_{i+1}, \ldots)\ \to\ (v_1, \ldots, v_{i-1}, e_i', e_{i+1}, \ldots)}\ \textsc{TupCong}$$

Constructor arguments reduce left-to-right via the existing App1/App2 rules (§4); constructor application is curried.

11 Trait System

Traits provide bounded polymorphism. A trait $T[\alpha]$ declares a family of methods; an `impl` provides those methods for a concrete type.

Trait declaration.

$$\texttt{trait}\ T\ \alpha = \left\{\ \texttt{fn}\ m_1 : \tau_1[\alpha]\ ;\ \cdots\ ;\ \texttt{fn}\ m_k : \tau_k[\alpha]\ \right\}$$

registers $m_1, \ldots, m_k$ in the global trait environment Φ.

Implementation.

$$\texttt{impl}\ T\ \tau_0\ \texttt{for}\ T_0 = \left\{\ \texttt{fn}\ m_i(\ldots) = e_i\ \right\}$$

provides each method m_i for the concrete type T_0. After type-checking, (m_i, T_0) is entered in the dispatch table $\mathcal{D} : (\mathsf{Name} \times \mathsf{Type}) \to \mathsf{Closure}$.

Bounded functions. A function may impose trait constraints on type variables:

$$\texttt{fn}\ f\ [T_1\, \alpha_1, \ldots, T_n\, \alpha_n](x_1 : \sigma_1, \ldots) : \rho = e$$

The body is checked in Γ extended with $\alpha_i \in T_i$ for each bound. Within the body, any method m of T_i may be applied to a term of type α_i.

Method dispatch. At call time the concrete type T_0 of the receiver is determined and $\mathcal{D}(m, T_0)$ is looked up. The compiler generates one dispatch stub per trait method; the stub inspects the constructor tag of the argument and branches to the appropriate implementation.

Remark 3. Dispatch stubs operate on ADT constructor tags. Primitive types (Int, Float, Bool, Str) carry no heap tag; trait dispatch for these types is routed through named runtime stubs rather than the tag-checking path.

12 Module System

Each `.lark` source file defines exactly one module.

Structure. A file begins with its module declaration, followed by any import declarations, then top-level declarations:

$$\begin{aligned}&\texttt{module}\ M\\&\texttt{import}\ N\ \texttt{exposing}\ (x_1, \ldots, x_k)\\&\mathit{decl}_1\ \ \cdots\ \ \mathit{decl}_n\end{aligned}$$

Top-level declarations may be prefixed with `export` to make them visible to importing modules.

Scoping. The type checker processes modules in dependency order. Importing module N from module M prepends the exported declarations of N to M's declaration list before type-checking. Imported function bodies are therefore visible to all subsequent passes: closure conversion, TAC lowering, and assembly.

Restrictions. Circular imports are not supported. Every name used in a declaration must be declared in the same file or imported explicitly.

13 Surface Grammar

The complete Lark grammar in ISO EBNF. `name` begins with a lowercase letter; `Name` with uppercase; `_` is a reserved wildcard token. In EBNF, `{ x }` denotes zero or more repetitions of x; `[ x ]` denotes an optional x; quoted strings are literal tokens.

Program structure

```
program     = module_decl { import_decl } { top_decl } ;
module_decl = "module" Name ;
import_decl = "import" Name [ "exposing" "(" name_list ")" ] ;
name_list   = export_name { "," export_name } ;
export_name = name | Name ;
top_decl    = [ "export" ] decl ;
decl        = fn_decl | let_decl | type_decl | trait_decl | impl_decl ;
```

Declarations

```
fn_decl      = "fn" name [ "[" bounds "]" ] "(" [ params ] ")"
               [ ":" type ] "=" expr ;
let_decl     = "let" name [ ":" type ] "=" expr ;
type_decl    = "type" Name { name } "=" type_body ;
trait_decl   = "trait" Name { name } "=" "{" { trait_method } "}" ;
impl_decl    = "impl" Name { atom_type } "for" atom_type
               "=" "{" { impl_method } "}" ;
params       = param { "," param } ;  param = name [ ":" type ] ;
bounds       = bound { "," bound } ;  bound = Name name ;
trait_method = "fn" name ":" type ;
impl_method  = "fn" name "(" [ params ] ")" "=" expr ;
```

Expressions

```
expr        = let_expr | if_expr | match_expr | lambda_expr | infix_expr ;
let_expr    = "let" name [ ":" type ] "=" expr "in" expr ;
if_expr     = "if" expr "then" expr "else" expr ;
match_expr  = "match" expr "with" { "|" pattern "=>" expr } "end" ;
lambda_expr = "fn" "(" [ params ] ")" "=>" expr ;
infix_expr  = unary_expr { op unary_expr } ;
unary_expr  = unary_op unary_expr | apply_expr ;
unary_op    = "-" | "not" ;
apply_expr  = atom { "(" [ args ] ")" } ;
args        = expr { "," expr } ;
atom        = literal | name | Name | "(" expr ")"
            | "(" expr "," expr { "," expr } ")" ;
```

Types and patterns

```
type      = fn_type ;
fn_type   = atom_type "->" fn_type | atom_type ;
atom_type = Name | Name "(" type { "," type } ")" | name | "()"
          | "(" type ")" | "(" type "," type { "," type } ")" ;
type_body = type | "|" Name [ "of" type ] { "|" Name [ "of" type ] } ;

pattern   = "_" | literal | name
          | Name [ "(" pattern { "," pattern } ")" ]
          | "(" pattern "," pattern { "," pattern } ")" ;
```

Operators and literals

```
op      = "+" | "-" | "*" | "/" | "==" | "!="
        | "<" | "<=" | ">" | ">=" | "and" | "or" ;
literal = integer | float | string | "true" | "false" | "()" ;
integer = digit { digit } ;
float   = digit { digit } "." digit { digit } ;
string  = '"' { str_char } '"' ;
```

Lexical

```
name = lowercase { letter | digit | "_" } ;
Name = uppercase { letter | digit | "_" } ;
```

Keywords (not valid as `name` or `Name`): `and else end export false fn if impl import in let match module not of or then trait true type where with`.

Part III Addendum: Phase 8 Rigor Extensions

This part specifies the Phase 8 extensions (snapshot `08/`). Like Part II these are paper rules; unlike Part II each rule here is enforced by an executable check in `08/src/` and exercised by the differential test suite across all four backends. Machine-checking them in the MLTT kernel is future work.

14 Defined Integer Semantics

Through Phase 7 the behaviour of Int arithmetic outside $[-2^{31}, 2^{31})$ was backend-dependent (the Python evaluators used unbounded integers; the RV32 backend wrapped). Phase 8 *defines* it:

Definition 1 (Int values)**.** Int values are the two's-complement 32-bit integers $\mathbb{Z}_{32} = \{-2^{31}, \ldots, 2^{31} - 1\}$. The wrap function $\mathsf{w} : \mathbb{Z} \to \mathbb{Z}_{32}$ is

$$\mathsf{w}(n) \;=\; \big((n + 2^{31}) \bmod 2^{32}\big) - 2^{31}.$$

Arithmetic. For $n_1, n_2 \in \mathbb{Z}_{32}$ and $\oplus \in \{+, -, \times\}$, the reduction rule for primitive application computes in $\mathbb{Z}$ and wraps:

$$n_1 \oplus n_2 \;\to\; \mathsf{w}(n_1 \oplus_{\mathbb{Z}} n_2).$$

Division and remainder. Division truncates toward zero and follows the RISC-V M-extension conventions, so no arithmetic operation is a stuck term:

$$n_1 \,/\, n_2 \;\to\; \begin{cases} -1 & n_2 = 0 \\ -2^{31} & n_1 = -2^{31},\ n_2 = -1 \\ \mathsf{trunc}(n_1/n_2) & \text{otherwise} \end{cases} \qquad n_1 \,\%\, n_2 \;\to\; \begin{cases} n_1 & n_2 = 0 \\ 0 & n_1 = -2^{31},\ n_2 = -1 \\ n_1 - n_2 \cdot \mathsf{trunc}(n_1/n_2) & \text{otherwise} \end{cases}$$

Remark 4. Wrap was chosen over trap because the compilation target decides: RV32 hardware wraps for free, so wrapping is the only semantics every backend can implement at zero cost. The consequence is visible in §16: integer descent is not well-founded under wrapping, and the totality judgment must refuse it.

15 Exhaustiveness

Phase 8 strengthens the MATCH typing rule of §9 with a coverage side condition, checked by Maranget's usefulness algorithm (`08/src/exhaust.py`).

Definition 2 (Pattern matrix, specialisation, default)**.** Let P be an $m \times n$ matrix of patterns. For a constructor c of arity k, the *specialisation* $\mathcal{S}(c, P)$ keeps each row whose first pattern is $c(p_1, \ldots, p_k)$ (replacing it by $p_1, \ldots, p_k$) or a wildcard/variable (replacing it by k wildcards), and drops the rest. The *default* $\mathcal{D}(P)$ keeps the rows whose first pattern is a wildcard/variable, minus their first column.

Definition 3 (Usefulness, coverage)**.** $\mathcal{U}(P, \vec{q})$ holds iff some value vector matches $\vec{q}$ but no row of P. A match on scrutinee type τ with arm patterns $p_1, \ldots, p_m$ is *exhaustive* iff $\neg\mathcal{U}\big((p_1)\cdots(p_m)^T,\ (\_)\big)$, decided by structural recursion on $\mathcal{S}$ and $\mathcal{D}$, driven by the column type's *complete signature*: the declared constructors for an ADT, $\{(),\}$ for Unit, $\{\texttt{true}, \texttt{false}\}$ for Bool, the single n-tuple constructor for products, and *no* finite signature for Int, Float, Str, IO, functions, and type variables (these require a wildcard or variable arm).

Strengthened rule.

$$\frac{\Gamma \vdash e : \tau \qquad \Gamma,\ \mathsf{binds}(p_i : \tau) \vdash e_i : \sigma \text{ for each } i \qquad \mathsf{exhaustive}(\tau;\ p_1, \ldots, p_m)}{\Gamma \vdash \texttt{match}\ e\ \texttt{with}\ \overline{\mid\ p_i \Rightarrow e_i}\ \texttt{end} : \sigma}\ \textsc{Match}^{+}$$

Theorem 2 (Match progress, paper)**.** *If a match expression is well typed under* MATCH^{+} *and its scrutinee is a value v, then* $\mathsf{match}(v, p_j)$ *succeeds for some arm j: the* MATCHARM *rule of §10 always applies, and the match-failure trap emitted by the compiler is unreachable.*

Remark 5. When the check fails, the usefulness recursion has *constructed* a value vector no arm matches; it is reported as a witness pattern (e.g. `Cons(Err(_), Nil)`), turning the rejection into a diagnosis.

16 Totality

A function may opt into a termination check: `fn total` $f(x_1{:}\tau_1, \ldots, x_n{:}\tau_n) = e$. The modifier is contextual — `total` remains a valid identifier.

Definition 4 (Structural order)**.** Write $y \prec x$ when y is bound at depth ≥ 1 inside a constructor or tuple pattern matched against x, or against some $z \prec x$. The relation is well-founded: every Lark value is a finite tree.

Definition 5 (Totality judgment)**.** $\mathsf{total}(f)$ holds iff

1. some fixed argument position i decreases at *every* recursive call $f(a_1, \ldots, a_n)$ in e: the argument a_i is a variable with $a_i \prec x_i$;
2. every other name referenced in e is known to terminate: a builtin, a constructor, a parameter of f (totality is *relative* — the caller supplies total arguments), another `total` function, or a function whose call graph reaches no cycle;
3. f itself appears only in call position, and f is not part of a mutual-recursion cycle (a v1 restriction; lexicographic and size-change measures are future work).

Theorem 3 (Relative termination, paper)**.** *If* $\mathsf{total}(f)$ *and f is applied to values whose function-typed components are themselves terminating, evaluation of the application terminates.*

Remark 6. $f(n-1)$ on Int is *not* accepted as decreasing, and this is soundness rather than incompleteness: under the wrapping semantics of §14, descent from a negative n never reaches a base case at 0 — it wraps through -2^{31}. The two Phase 8 semantic decisions are mutually constraining: defined overflow is what makes integer descent genuinely ill-founded, and structural descent on data is the discipline that remains.